# Service Overview

## ภาพรวมของบริการ (Service Overview)

## Manage Dispatch Service

**Github:** https://github.com/Slasherw/ManageDispatchService

### 1. Service Owner
นายนพกรณ์ สงเคราะห์ รหัสนักศึกษา 6609540023

### 2. Service Purpose
Manage Dispatch Service เป็นบริการที่รับผิดชอบการบริหารจัดการคำสั่งการกู้ภัย โดยทำหน้าที่เชื่อมโยงทีมกู้ภัย (Rescue team) ที่เฉพาะเจาะจงเข้ากับคำร้องขอความช่วยเหลือ (Request) รวมถึงติดตามสถานะการร้องขอทั้งหมดที่เกิดขึ้นในเหตุการณ์ภัยพิบัติ เพื่อให้ส่วนกลางสามารถมอนิเตอร์สถานะการทำงานได้อย่างมีประสิทธิภาพ

### 3. Pain Point ที่แก้ไข
ในช่วงเหตุการณ์ภัยพิบัติที่มีความโกลาหล ศูนย์สั่งการ (Dispatchers) มักจะสูญเสียการติดตามสถานะของหน่วยกู้ภัยที่ถูกส่งออกไปปฏิบัติหน้าที่ ซึ่งปัญหาดังกล่าวนำไปสู่การส่งทีมกู้ภัยไปทำงานซ้ำซ้อนในพื้นที่เดิม หรือทำให้มีบางพื้นที่ที่ต้องการความช่วยเหลือถูกละเลยไป

### 4. Target Users
- เจ้าหน้าที่ศูนย์สั่งการ / ผู้สั่งการ (Dispatcher)
- บริการนี้ทำหน้าที่เป็น Backend Service หลักที่เชื่อมต่อระหว่าง Request Service, Team Service และ Mission Progress Service

### 5. Service Boundary
In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ):
- จัดเก็บและจัดการข้อมูลคำสั่งการ (Dispatch Records)
- จัดการสถานะการตอบรับภารกิจ (Mission Acceptance Status) ตั้งแต่เริ่มต้น (PENDING) ไปจนถึงรับภารกิจ (ACCEPT) หรือยกเลิก (DECLINE)
- บังคับใช้ตรรกะการตรวจสอบความพร้อมของทีมกู้ภัยก่อนลงพื้นที่

Out-of-scope / Not Responsible For (ไม่รับผิดชอบ):
- การจัดการข้อมูลพื้นฐานของทีมกู้ภัย (Team Master Data)
- การจัดการรายละเอียดเชิงลึกของคำร้องขอความช่วยเหลือจากประชาชน (Request/Incident Master Data)

### 6. Autonomy / Decision Logic
บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:
- การตรวจสอบและตัดสินใจอนุมัติคำสั่งการ โดยอิงจากสถานะ "AVAILABLE" ของทีมกู้ภัยจาก Team Service
- การปฏิเสธการสร้างคำสั่งการ (Dispatch) หากอ้างอิงถึงข้อมูลที่ไม่มีอยู่จริง (เช่น ไม่มี Incident ID) เพื่อป้องกันการเกิดข้อมูลกำพร้า (Orphan data) ในระบบ

- การจัดการสถานะ Flow การทำงาน เช่น การเปลี่ยนสถานะจาก PENDING เป็น ACCEPT หรือ DECLINE

**7. Owned DataDispatch Records:**

- ข้อมูลหลักของการสั่งการ เช่น dispatch_id, เวลาที่ส่งทีมออกไป (dispatched_at), เวลาที่ทีมไปถึงที่หมาย (arrival_time), และบันทึกหลังจบภารกิจ(completion_note) บริการนี้ต้องเป็นเจ้าของข้อมูลเหล่านี้เพื่อใช้ควบคุมการทำงาน
- Mission Acceptance State: สถานะปัจจุบันของการสั่งการ ที่เปลี่ยนแปลงโดยการรับทำภารกิจหรือปฏิเสธภารกิจของ Rescue Team

**8. Linked Data (Reference Only)**

- request_id อ้างอิงจาก Request Service
- team_id อ้างอิงจาก Team Service
- mission_status อาจมีการดึงหรือเชื่อมโยงจาก Mission Progress Service

**9. Non-Functional Requirements**

- ระบบต้องมีกลไกป้องกันข้อมูลตกหล่น (Idempotency) และไม่สร้าง Dispatch record ซ้ำซ้อน หาก Request ID ไม่มีอยู่จริงในระบบ ระบบต้องตอบกลับด้วย HTTP 404 Not Found ทันที
- ต้องมีการจัดการข้อผิดพลาด (Failure handling) ที่ดี หากไม่สามารถดึงข้อมูลสถานะทีมจาก Team Service ได้

# Sync Contract

# Synchronous Function Contract

# Base URL: http://TBD.com

API Contract 1: Create Dispatch Order (สร้างคำสั่งการและรอการตอบรับ)

ข้อมูลทั่วไป

- Name: Create Dispatch Order for assign Rescuer to request
- Method: POST
- Path: /dispatches
- Type: Syncronous

คำอธิบาย
สร้างบันทึกคำสั่งการใหม่ เพื่อส่งงานไปให้ทีมกู้ภัย (ระบบจะตั้งสถานะเริ่มต้นเป็น PENDING)

**Request**

**Query Params**

- ไม่มี

**Headers**

- Accept: application/json
- Content-Type: application/json

**Body:**

{ "requestId": "REQ-10293", "teamId": "TEAM-005", "priorityLevel": 1 }

Response

Created: 201

{
 "dispatch_id": "DSP-88472",
"request_id": "REQ-10293",
 "team_id": "TEAM-005",
"status": "PENDING",
"dispatched_at": "2026-03-03T14:30:00Z"
}

Error: 400

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "requestId and teamId required",
    "traceId": "uuid"
  }
}
```

Error: 404

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "teamId or requestId not found",
    "traceId": "uuid"
  }
}
```

**Dependency / Reliability**

- เรียก Team Service เพื่อตรวจสอบสถานะทีมกู้ภัย (AVAILABLE)
- เรียก Request Service เพื่อตรวจสอบความถูกต้องของ request_id
- ไม่ Idempotent (Not Idempotent)
- Timeout: 30s

## API Contract #2: Get Dispatches by Team

**ข้อมูลทั่วไป**

- Name: Get dispatches by team
- Method: GET Path: /v1/dispatches
- Type: Synchronous

**คำอธิบาย**

ใช้ดึงข้อมูลคำสั่งการที่ถูกมอบหมายให้กับทีมกู้ภัยแต่ละทีม เพื่อให้บริการอื่น (เช่น แอปพลิเคชันของกู้ภัย) ใช้แสดงผลว่ามีภารกิจไหนที่กำลังรอการตอบรับ (PENDING) หรือรับงานไปแล้ว (ACCEPT)

**Request Query Params**

- teamId (string, required)

- status (string, optional)

**Headers**

- Accept: application/json

**Body:** ไม่มี

**Response**

**Success: 200**


```
{
  "teamId": "TEAM-005",
  "items": [
    {
      "dispatchId": "DSP-88472",
      "requestId": "REQ-10293",
      "status": "PENDING",
      "priorityLevel": 1,
      "dispatchedAt": "2026-03-03T14:30:00Z"
    }
  ]
}
```


**Error: 400**

JSON


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "teamId required",
    "traceId": "uuid"
  }
}
```


**Dependency / Reliability**

- ไม่เรียก service อื่น
- Idempotent
- Timeout: 30s

## API Contract #3: Update Mission Acceptance Status

**ข้อมูลทั่วไป**

- Name: Update mission acceptance status
- Method: PATCH
- Path: /v1/dispatches/{dispatchId}/
- status Type: Synchronous

**คำอธิบาย**

ใช้สำหรับให้ทีมกู้ภัยอัปเดตสถานะการตอบรับภารกิจ โดยเปลี่ยนจาก PENDING เป็น ACCEPT (รับงาน) หรือ DECLINE (ปฏิเสธงานพร้อมระบุเหตุผล)

**Request**

**Query Params** ไม่มี

**Headers**

- Accept: application/json
- Content-Type: application/json

**Body:**

{ "status": "ACCEPT", "note": "" }

**Response**

**Success: 200**

{

  "dispatchId": "DSP-88472",

  "status": "ACCEPT",

  "updatedAt": "2026-03-03T14:35:00Z"

}

**Error: 400**

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "invalid status value (must be ACCEPT or DECLINE)",
    "traceId": "uuid"
  }
}
```


**Error: 409**


```
{
  "error": {
    "code": "CONFLICT_ERROR",
    "message": "dispatch order already accepted or declined",
    "traceId": "uuid"
  }
}
```


**Dependency / Reliability**

- ไม่เรียก service อื่น
- Idempotent
- Timeout: 30s

# Async Contract

# Asynchronous Function Contract

## Message Contract #1: Request Team Dispatch

### ข้อมูลทั่วไป

- **Message Name:** `DispatchTeamRequested`
- **Interaction Style:** Request–Async Response (via Message Broker)
- **Producer:** Request Service (หรือ Incident Service)
- **Consumer:** Manage Dispatch Service
- **Channel/Queue:** `dispatch.commands.v1`
- **Version:** v1

### คำอธิบาย

ส่งคำขอให้จ่ายงาน (dispatch) ทีมกู้ภัยไปยังคำร้องขอความช่วยเหลือแบบ asynchronous โดยผู้ส่ง (Producer) จะได้รับแจ้งผลลัพธ์การสร้างคำสั่งการกลับไปภายหลังผ่าน result event ที่เชื่อมโยงด้วย correlationId

### Request

#### Message Headers (required unless stated)

- `messageType`: DispatchTeamRequested
- `messageId`: UUID (ใช้เป็น idempotency key ของคำขอ)
- `replyTo`: `request.dispatch.results.v1`
- `sentAt`: ISO-8601 datetime
- `traceId`: string/uuid (optional)

### Message Body

```
{
  "requestId": "REQ-10293",
  "teamId": "TEAM-005",
  "priorityLevel": 1,
  "requestedBy": "RequestService"
}
```

### Field Definition:

| Field | Type | Require | Description |
|---|---|---|---|
| `requestId` | string | Y | รหัสคำร้องขอความช่วยเหลือจาก<br><br>Request Service |
| `teamId` | string | Y | รหัสทีมกู้ภัยเป้าหมายที่ต้องการส่งไป |
| `priorityLevel` | integer | Y | ระดับความสำคัญของงาน (เช่น 1=สูงมาก, 2=ปานกลาง, 3=ต่ำ) |
| `requestedBy` | string | Y | ชื่อ service ผู้ส่งคำขอ |

**Validation Rules**

- `requestId` และ `teamId` ต้องไม่เป็นค่าว่าง
- `priorityLevel` ต้องเป็นจำนวนเต็มและมากกว่า 0
- `messageId` ต้องไม่ซ้ำกับ message ที่เคยประมวลผลแล้ว (idempotency check ป้องกันการส่งทีมซ้ำซ้อน)

---

**Response Message Headers (required unless stated)**

- `messageType`: DispatchOrderCreated (กรณีสำเร็จ) หรือ DispatchTeamRejected (กรณีล้มเหลว)
- `messageId`: UUID ของข้อความนี้
- `correlationId`: request.messageId (เชื่อมโยงกับคำขอต้นทาง)
- `sentAt`: ISO-8601 datetime
- `traceId`: string/uuid (optional)

**Success Message Body**

```
{
  "requestId": "REQ-10293",
  "teamId": "TEAM-005",
```

"dispatchId": "DSP-88472",
"status": "PENDING",
"dispatchedAt": "2026-03-03T14:30:00Z"
}

**Reject/Error Message Body**

{
"requestId": "REQ-10293",
"teamId": "TEAM-005",
"status": "REJECTED",
"reasonCode": "TEAM_NOT_AVAILABLE",
"reasonMessage": "The requested rescue team is currently not available or engaged in another mission."
}

**Field Definition:** |

| Field | Type | Require | Description |
|---|---|---|---|
| `requestId` | string | Y | รหัสคำร้องขอความช่วยเหลือจากระบบต้นทาง |
| `teamId` | string | Y | รหัสทีมกู้ภัยเป้าหมาย |
| `dispatchId` | string | N | รหัสคำสั่งการที่ถูกสร้างขึ้นโดย Manage Dispatch Service (มีเฉพาะกรณีสำเร็จ) |
| `status` | enum | Y | สถานะผลลัพธ์ของคำขอ (PENDING / REJECTED) |
| `dispatchedAt` | datetime | N | เวลาที่ทำการสร้างคำสั่งการสำเร็จ |

| | | | |
|---|---|---|---|
| `reasonCode` | string | N | รหัสเหตุผลที่เป็น machine-readable ใช้สำหรับระบบอื่นตัดสินใจต่อ (มีเฉพาะกรณี REJECTED) |
| `reasonMessage` | string | N | ข้อความอธิบายแบบ human-readable เพื่อใช้แสดงผลใน log หรือช่วย debug (มีเฉพาะกรณี REJECTED) |

**Validation Rules**

- `correlationId` ต้องตรงกับ `messageId` ของ request ที่เคยได้รับ
- ถ้า `status = PENDING` ต้องมี `dispatchId` และ `dispatchedAt`
- ถ้า `status = REJECTED` ต้องมี `reasonCode` และ `reasonMessage`
- `reasonCode` ต้องเป็นค่าที่กำหนดไว้ล่วงหน้า (เช่น `TEAM_NOT_AVAILABLE`, `REQUEST_NOT_FOUND`) ไม่ใช่ free text

# Service Data

## Service Data

### 1) Dispatch Records (Owned by this service)

ข้อมูลหลักของการสั่งการ ควบคุมว่าส่งทีมไหน ไปทำภารกิจใด และจบงานตอนไหน

| Field Name | Type | Required | Description | Example |
|---|---|---|---|---|
| dispatchId | string | Y (Primary Key) | รหัสคำสั่งการ | DSP-88472 |
| requestId | string | Y (Foreign Key) | อ้างอิงคำร้องขอความช่วยเหลือจาก Request Service | REQ-10293 |
| teamId | string | Y (Foreign Key) | อ้างอิงทีมกู้ภัยจาก Team Service | TEAM-005 |
| priorityLevel | integer | Y | ระดับความสำคัญของงาน (1=สูงมาก) | 1 |
| dispatchedAt | datetime | Y | เวลาที่สร้างคำสั่งการ/มอบหมายงาน | 2026-03-03T14:30:00Z |

| | | | | |
|---|---|---|---|---|
| arrivalTime | datetime | N | เวลาที่ทีมกู้ภัยเดินทางไปถึงจุดเกิดเหตุ | 2026-03-03T15:00:00Z |
| completionNote | text | N | บันทึกรายงานจากทีมกู้ภัยหลังจบภารกิจ | ช่วยเหลือผู้บาดเจ็บ 2 รายสำเร็จ |
| idempotencyKey | uuid | Y | ป้องกันการสร้างคำสั่งการซ้ำซ้อน | f47ac10b-... |
| createdAt | datetime | Y | เวลาที่สร้างข้อมูลนี้ในระบบ | 2026-03-03T14:30:00Z |

## 2) Mission Acceptance Status (Owned by this service)

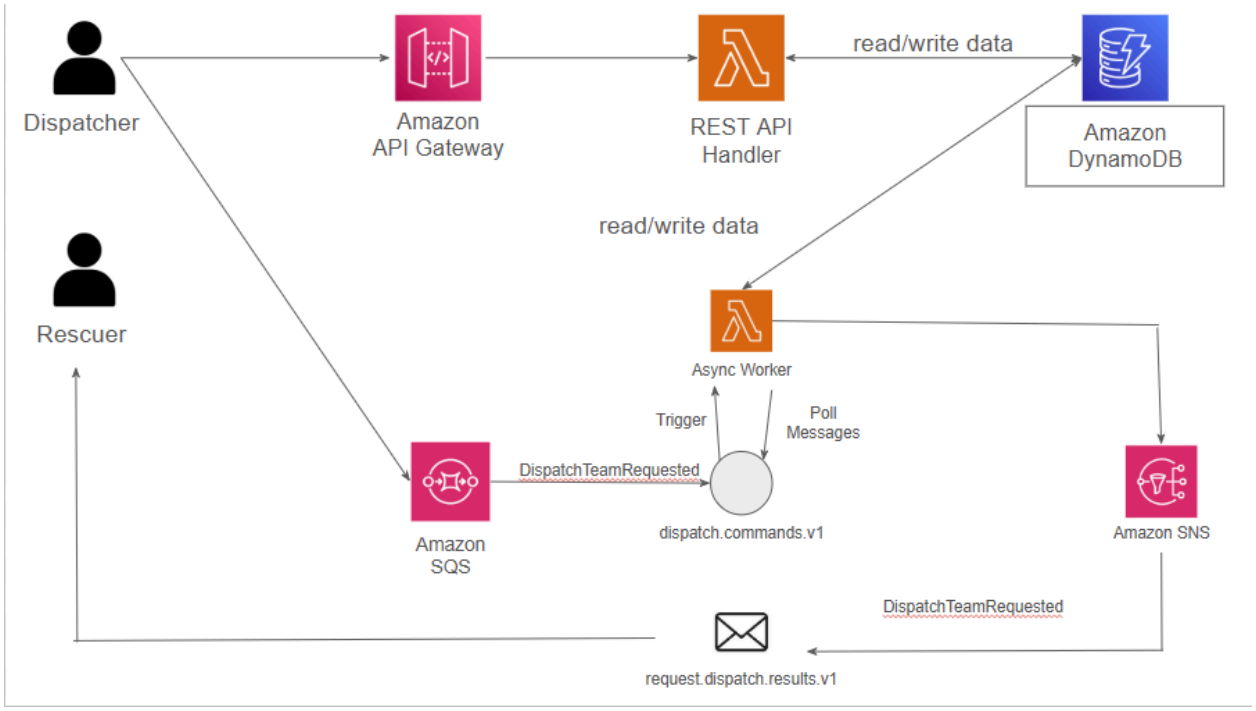
ข้อมูลสถานะปัจจุบันของการสั่งการ ที่อัปเดตโดยการตอบรับหรือปฏิเสธของ Team กู้ภัยหน้างาน

| Field Name | Type | Required | Description | Example |
|---|---|---|---|---|
| dispatchId | string | Y (Foreign Key) | อ้างอิงรหัสคำสั่งการ (Dispatch Record) | DSP-88472 |
| status | enum | Y | PENDING / ACCEPT / DECLINE | ACCEPT |

| | | | | |
|---|---|---|---|---|
| statusNote | string | N | เหตุผลหรือหมายเหตุประกอบการเปลี่ยนสถานะ | ทีมพร้อมเดินทางภายใน 5 นาที |
| updatedBy | string | N | รหัสผู้ใช้งานหรือระบบที่เปลี่ยนสถานะ (เช่น รหัสหัวหน้าทีม) | USER-992 |
| lastUpdatedAt | datetime | Y | เวลาที่มีการอัปเดตสถานะล่าสุด | 2026-03-03T14:35:00Z |

# Service Architecture

## Service Architecture




**Components**

- **User/Client**: ผู้เรียกใช้งานระบบ ได้แก่ เจ้าหน้าที่ศูนย์สั่งการ (Dispatcher) ที่ใช้งานผ่าน Web Dashboard และทีมกู้ภัย (Rescue Team) ที่ใช้งานผ่าน Mobile Application
- **API Gateway**: จุดรับคำขอแบบ Synchronous สำหรับการเรียกดูข้อมูลรายการสั่งการ (GET) และการอัปเดตสถานะการรับภารกิจ (PATCH)
- **Lambda REST API Handler**: ทำหน้าที่ประมวลผล Logic ในการดึงข้อมูลจากฐานข้อมูล หรือตรวจสอบเงื่อนไขก่อนเปลี่ยนสถานะภารกิจจาก PENDING เป็น ACCEPT หรือ DECLINE
- **DynamoDB**: ฐานข้อมูล NoSQL ที่จัดเก็บข้อมูลที่ Manage Dispatch Service เป็นเจ้าของ ได้แก่ Dispatch Records และ Mission Acceptance State
- **Amazon SQS (dispatch.commands.v1)**: คิวรับคำขอสั่งการ (Dispatch Request) แบบ Asynchronous ที่ส่งมาจาก Request Service
- **Lambda Async Worker**: ทำการ Poll ข้อความคำขอสั่งการจาก SQS เพื่อตรวจสอบความพร้อมของทีมกู้ภัย และบันทึกข้อมูลการสั่งการลงใน DynamoDB
- **Amazon SNS**: ช่องทางประกาศผลลัพธ์การสร้างคำสั่งการ (Dispatch Result) เพื่อแจ้งกลับไปยัง Service ต้นทางที่ร้องขอ
- **Subscriber (request.dispatch.results.v1)**: บริการที่รอรับผลลัพธ์ เช่น Request Service เพื่ออัปเดตสถานะในส่วนของตนเองต่อไป

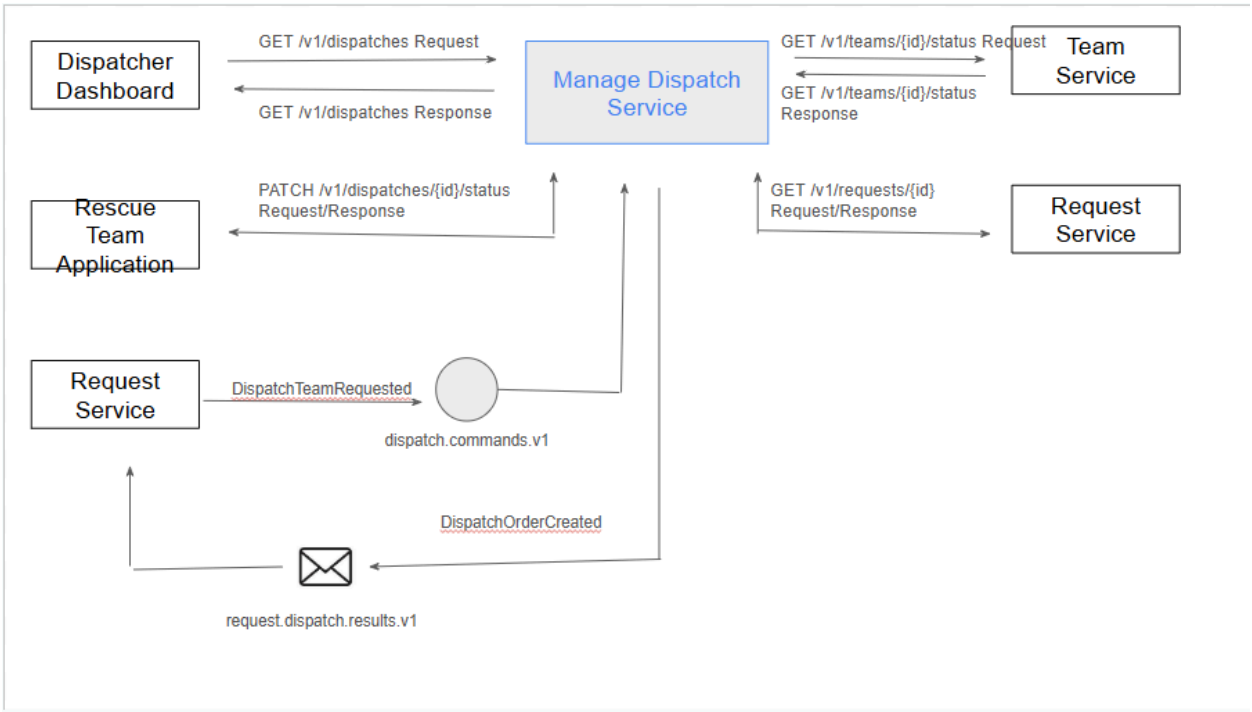
**Explanation**

สถาปัตยกรรมของ Manage Dispatch Service ประกอบด้วยสองรูปแบบการทำงานหลัก:

1. **การสื่อสารแบบ Synchronous**: ผู้ใช้งาน (Dispatcher หรือ Rescue Team) จะเรียกใช้งาน API ผ่าน **Amazon API Gateway** เพื่อสอบถามสถานะภารกิจหรืออัปเดตการตอบรับงาน ซึ่งคำขอจะถูกส่งต่อไปยัง **AWS Lambda** เพื่อตรวจสอบความถูกต้องและอ่าน/เขียนข้อมูลลงใน **Amazon DynamoDB** โดยตรง รูปแบบนี้ช่วยให้ทีมกู้ภัยสามารถยืนยันการรับงานได้ทันที
2. **การสื่อสารแบบ Asynchronous**: เมื่อมีการร้องขอให้ส่งทีมกู้ภัยจากระบบอื่น ระบบจะส่งข้อความมายัง **Amazon SQS** ซึ่งทำหน้าที่เป็น Buffer รับงานไว้ไม่ให้ข้อมูลสูญหายแม้ในช่วงที่มีโหลดสูง จากนั้น **Lambda Async Worker** จะดึงงานไปประมวลผลเพื่อสร้างเรคคอร์ดการสั่งการ และประกาศผลลัพธ์กลับผ่าน **Amazon SNS** เพื่อให้ระบบต้นทางรับทราบสถานะการจองทีมกู้ภัยโดยไม่ต้องรอการประมวลผลจนเสร็จสิ้น ช่วยเพิ่มความทนทานและความน่าเชื่อถือให้กับระบบในภาวะภัยพิบัติที่มีการร้องขอเข้ามาจำนวนมาก

# Service Interaction

## Service Interaction




**Upstream Services (บริการต้นทางที่เรียกใช้งาน Manage Dispatch Service)**

- **Dispatcher Dashboard Service**
  - เรียกใช้งาน `GET /v1/dispatches` แบบ **synchronous** เพื่อดึงข้อมูลรายการคำสั่งการและสถานะการปฏิบัติงานของทีมกู้ภัยทั้งหมด สำหรับนำไปแสดงผลบนหน้าจอควบคุมและสนับสนุนการตัดสินใจของผู้สั่งการ
- **Request Service**
  - ส่งคำขอสั่งการทีมกู้ภัยในรูปแบบ event `DispatchTeamRequested` ผ่าน command queue (`dispatch.commands.v1`) แบบ **asynchronous** เพื่อมอบหมายงานกู้ภัยไปยังพื้นที่เกิดเหตุโดยไม่รอผลลัพธ์ทันที
- **Rescue Team Application**
  - เรียกใช้งาน `PATCH /v1/dispatches/{dispatchId}/status` แบบ **synchronous** เพื่อให้ทีมกู้ภัยในพื้นที่สามารถอัปเดตสถานะการตอบรับภารกิจ (ACCEPT/DECLINE) กลับมายังระบบส่วนกลางได้ทันที

**Downstream Services (บริการปลายทางที่ Manage Dispatch Service เรียกหรือส่งข้อมูลไป)**

- **Team Service**
  - ถูกเรียกใช้งานแบบ **synchronous** ผ่าน `GET /v1/teams/{teamId}/status` เพื่อยืนยันว่าทีมกู้ภัยมีตัวตนจริงและมีสถานะ "AVAILABLE" ก่อนที่ระบบจะดำเนินการสร้างเรคคอร์ดคำสั่งการ

- **Request Service**
    - ถูกเรียกใช้งานแบบ **synchronous** ผ่าน `GET /v1/requests/{requestId}` เพื่อยืนยันความถูกต้องและสถานะของคำร้องขอความช่วยเหลือก่อนทำการบันทึกข้อมูลการสั่งการ เพื่อป้องกันการเกิดข้อมูลที่ไม่มีแหล่งอ้างอิง (Orphan data)
- **Notification / Result Channel (`request.dispatch.results.v1`)**
    - รับ event `DispatchOrderCreated` หรือ `DispatchTeamRejected` ที่ส่งออกจาก Manage Dispatch Service เพื่อแจ้งผลลัพธ์ของการประมวลผลคำสั่งการกลับไปยัง Request Service หรือระบบที่เกี่ยวข้อง

# Dependency Mapping

## Dependency Mapping – Manage Dispatch Service

### 1. Team Service

- **Type:** Service
- **Interaction Style:** Synchronous (REST)
- **Purpose:** ตรวจสอบสถานะความพร้อมของทีมกู้ภัย (เช่น ต้องเป็น AVAILABLE) ก่อนดำเนินการมอบหมายภารกิจ
- **Criticality:** Critical
- **Failure Handling:** หาก Team Service ไม่ตอบสนอง จะปฏิเสธคำขอการ Dispatch ชั่วคราว และส่งผลลัพธ์เป็น REJECTED พร้อม `reasonCode = TEAM_SERVICE_UNAVAILABLE`

### 2. Request Service

- **Type:** Service
- **Interaction Style:** Synchronous (REST)
- **Purpose:** ตรวจสอบความถูกต้องและสถานะของรหัสคำร้องขอความช่วยเหลือ (`requestId`) ก่อนทำการบันทึกข้อมูลการสั่งการ
- **Criticality:** Critical
- **Failure Handling:** หากบริการไม่ตอบสนอง จะปฏิเสธการสร้างคำสั่งการทันทีเพื่อป้องกันการเกิดข้อมูลที่ไม่มีแหล่งอ้างอิง (Orphan Data) ในระบบ

### 3. Amazon SQS (dispatch.commands.v1)

- **Type:** Queue
- **Interaction Style:** Asynchronous (Command Queue)
- **Purpose:** รับคำสั่งมอบหมายภารกิจกู้ภัยแบบเป็นลำดับจาก Request Service หรือบริการอื่นที่เกี่ยวข้อง
- **Criticality:** Critical
- **Failure Handling:** * ใช้ Retry Policy ของ AWS Lambda ในการพยายามประมวลผลใหม่
    - หากการประมวลผลล้มเหลวซ้ำจนครบจำนวนครั้งที่กำหนด จะส่งข้อความไปยัง Dead Letter Queue (DLQ) เพื่อการตรวจสอบภายหลัง

### 4. Amazon SNS (request.dispatch.results.v1)

- **Type:** Topic
- **Interaction Style:** Asynchronous (Event Notification)
- **Purpose:** กระจายผลลัพธ์การสร้างคำสั่งการ (Dispatch Order Created/Rejected) ให้กับบริการต้นทางหรือระบบแจ้งเตือนอื่นๆ
- **Criticality:** Non-critical
- **Failure Handling:** หากการประกาศเหตุการณ์ (Publish) ไม่สำเร็จ ระบบจะบันทึก Log และทำการ Retry ตาม Policy ของ AWS โดยไม่ส่งผลกระทบต่อข้อมูลการสั่งการที่บันทึกสำเร็จแล้วในฐานข้อมูล

**5. Amazon DynamoDB**

- **Type:** Database
- **Interaction Style:** Internal Data Access
- **Purpose:** จัดเก็บข้อมูลหลักของบริการ ได้แก่ Dispatch Records และสถานะการตอบรับภารกิจ (Mission Acceptance State)
- **Criticality:** Critical
- **Failure Handling:** หากการเขียนข้อมูล (Write) ล้มเหลว ระบบจะทำการยับยั้งกระบวนการทั้งหมด และจะไม่ส่ง Event แจ้งผลลัพธ์ออกไปทาง SNS เพื่อรักษาความถูกต้องของข้อมูล (Data Consistency)